Contender 14 for BusyLean

Bi-Diagonal Rank Reflection over Dynamic-Kind System F_ω

Research memorandum for the BusyLean project

October 12, 2026

Abstract

BusyLean asks for closed, constructive natural numbers accompanied by formal strict comparisons in Lean. At the audited repository tip, Contender 13 is a single reflection over a predicative dynamic-kind extension of System F_ω, using Contender 12's construction as its oracle. This memorandum proposes Contender 14, a qualitatively stronger construction with three coupled layers. First, the dynamic-kind language is upgraded from one unary oracle to a computably packed countable oracle family. Second, transformer reflection is lifted one further type level to a hierarchy of meta-transformers. Successor meta-ranks are *anchored*: they preserve the preceding rank at a distinguished coordinate before autonomously lifting it, which yields a direct theorem that every successor rank is strictly larger. Third, both the meta-rank and the depth of the autonomous tree schedule are set to the previous contender's growth function at the current input, rather than to any fixed numeral. A final dynamic-kind reflection dominates the complete rank-diagonal family.

The proposed Lean sources prove that the complete Contender 13 family is pointwise below the new function and hence that Contender 13 is below Contender 14. They also dominate every enumerated program in the final dynamic-kind reflection layer. Every definition is structural or recursive on a natural number and contains none of Lean's noncomputable escape hatches. The accompanying package contains the Lean overlay, focused axiom and dependency audits, a clean-build harness, a static source audit, an integration patch, this source, and the compiled PDF. The PDF and static audits were produced in the present environment. Lean 4.34.0 was not installed here, so kernel replay and time-budget measurements remain an explicit integration step rather than a claimed result.

Research status

Baseline. The source audit targets commit `1ee50c7d97bbed57a60709f661453c3e213f4db3` of BusyLean; the commit is dated October 13, 2026 UTC, which is October 12 in the Pacific time zone. That HEAD changes only the repository documentation relative to parent commit `7f6a5c6e44ef31e389b10f6ab6da2bb0284c3502`; the Lean source blobs used by the construction are unchanged.

Completed here. Repository-level design audit; complete Lean source with no placeholders; source-level dependency and forbidden-token checks; LaTeX compilation; PDF rendering and visual inspection; reproducibility and integration scripts.

Not claimed here. Lean kernel acceptance, exact axiom output, and the 15-second/60-second contest timing budgets. The package makes those checks a single command on a machine with the repository's pinned Lean 4.34.0 toolchain.

Contents

1 The research question **3**

2 Audited baseline: Contenders 5–13 **3**

3 Design constraints **5**
3.1 No oracle monotonicity assumption . . . 5
3.2 The meta-rank must have extensional content . . . 5
3.3 The previous contender must enter as a construction, not a constant . . . 5
3.4 The closure must remain executable . . . 5
3.5 The final layer should expose a direct program bound . . . 5

4 The existing progressivity interface **6**

5 One level higher: meta-transformers **6**

6 Anchored successor ranks **7**

7 Family-aware dynamic-kind reflection **8**

8 The proposed number **8**

9 Formal comparison with Contender 13 **9**

10 Additional formal margins **10**
10.1 Every successor rank is strictly larger . . . 10
10.2 The diagonal rank escapes every fixed finite rank . . . 10
10.3 Final-layer program domination . . . 11

11 Why the increase is significant **11**

12 Computability and constructiveness **11**

13 Axiom and dependency discipline **12**

14 Implementation layout **13**

15 Validation status and likely proof obligations **13**
15.1 What was checked . . . 13
15.2 What kernel replay must confirm . . . 13
15.3 Performance risk . . . 14

16 Informal growth interpretation—and its limits **14**

17 Potential refinements after kernel replay **14**
17.1 Transitive rank comparison . . . 14
17.2 Coordinate-sensitive positive branches . . . 14
17.3 A hierarchy of dynamic universes . . . 14
17.4 Internal tree generation . . . 15
17.5 Exact ordinal analysis . . . 15

18 Reproduction protocol **15**

19 Conclusion **16**

A Complete Lean source: meta-reflection hierarchy **16**

B Complete Lean source: Contender 14 definition **18**

C Complete Lean source: comparison theorems **20**

D Focused audit source **22**

1 The research question

The large-number contest underlying BusyLean is not a contest in typographical compression. A successful entry must simultaneously satisfy four demands. The proposed number must be a closed natural number; it must be computable in principle; it must avoid semantic or implementation loopholes; and it must come with a formal theorem that it exceeds the previous accepted contender. The last condition is crucial. For numbers produced by normalization and reflection, decimal notation is irrelevant and numerical evaluation is not a reasonable validation strategy. The mathematical content is the construction and its order theorem.

The repository's rules follow the Lean edition of Cody Roux's *name-the-biggest-number* contest. The current policy permits only the axioms already used by the trusted implementation infrastructure, namely `propext` and `Quot.sound`; it disallows `Classical.choice`, unfinished proofs, native-decision shortcuts, opaque stand-ins, partial definitions, and external implementations. It also requires the new definition to avoid mentioning the previous *constant* itself. A prior construction may be reused as a family or oracle, because that is precisely how a genuinely stronger closure operation is built.

The immediate research problem is therefore:

> Construct a total Lean definition whose growth mechanism is qualitatively stronger than Contender 13, prove a pointwise comparison with Contender 13's underlying family, preserve the repository's no-monotonicity discipline for oracle reflection, and keep the proof modular enough to audit.

The proposed answer is a bi-diagonal hierarchy. "Bi-diagonal" means that the previous contender controls both (i) how many meta-level promotions are taken and (ii) how high the autonomous tree schedule is raised. The construction is then closed by one more reflection in the strongest object language currently formalized in the repository.

2 Audited baseline: Contenders 5–13

The repository is cumulative. Understanding what would count as a meaningful advance requires separating two sources of growth: the strength of the reflected object language and the strength of the amplifier surrounding it. Table 1 records the relevant architecture at the audited commit. The descriptions are architectural rather than claims of exact proof-theoretic calibration.

Table 1: Architecture of the accepted chain at the audited commit.

Entry	Reflected computational language	Amplifier or diagonal mechanism
C_5	Raw System T terms	Finite maximum over terms of syntax depth at most 42.

Entry	Reflected computational language	Amplifier or diagonal mechanism
C_6	System T with W-types over all finite types	Family reflection along every W-tree generated by the language's own index.
C_7	System F with an oracle	Certified normalization, finite syntax maximum, and an outer numerical diagonal.
C_8	System F_ω with an oracle	Reflection over type operators and an outer diagonal.
C_9	System F_ω with a packed oracle family	Forest reflection along the W-language's self-generated index.
C_{10}	Kind-polymorphic System F_ω	Reflection over programs that can select kinds of unbounded finite order at run time.
C_{11}	Kind-polymorphic System F_ω with a packed family	Forest reflection along the W-language index.
C_{12}	The C_{11} reflection transformer	Operator-level reflection along an autonomous hyper-tree stacked above the language-defined index.
C_{13}	Dynamic-kind System F_ω	One unary-oracle reflection over `family12`, followed by the standard outer double diagonal.

The last two entries are the essential baseline.

Contender 12. The repository defines a family transformer

$$\texttt{fkTransformer}\ fs\ o\ n = \texttt{reflect11}\ fs\ n,$$

then iterates transformer lifting along an autonomous W-tree

$$\texttt{tree12} = \texttt{realize}\big(\texttt{hyper}\ 42\ (\texttt{encode}\ \texttt{index})\big).$$

The function `operatorStage` is structurally recursive on that tree. Progressivity is proved without assuming that reflection is monotone in its oracle. Instead, each reflected output explicitly exceeds every visible seed coordinate.

Contender 13. The object language is extended with a dynamic kind interpreted predicatively as a dependent sum of semantic values over dyn-free kinds. It supports packing and unpacking and two levels of kind quantification. The reflection function is

$$\begin{aligned}\texttt{reflectFd}\ f\ n = n + 1 + \max\Big(&f(n),\\ &\max\{\texttt{fvalue}\ f\ n\ t : t \in \texttt{ctmsUpTo}\ n\}\Big).\end{aligned}$$

Then

$$\texttt{family12}(n) = F_{12}(F_{12}(n)), \qquad F_{13}(n) = \texttt{reflectFd}\ \texttt{family12}\ n,$$

and

$$C_{13} = F_{13}(F_{13}(42)).$$

The comparison theorem follows from two explicit inequalities: the oracle is below its reflection, and every input is below its reflection.

Contender 13 is stronger than Contender 12 because it changes the object language. It does not, however, reapply Contender 12's operator-level amplification to the new dynamic-kind language. That

gap is the starting point for Contender 14, but merely copying `operatorStage` once would leave a fixed amplifier. The present proposal instead constructs an unbounded, strictly increasing hierarchy above that level.

3 Design constraints

Five constraints shaped the construction.

3.1 No oracle monotonicity assumption

A reflected program may inspect an oracle value and branch on it. Consequently, pointwise larger oracle values need not produce pointwise larger program outputs. Any proof based on monotonicity of the semantic evaluator would be unsound. BusyLean's existing reflection layers avoid this by incorporating visible oracle values directly into a finite maximum. Contender 14 preserves that discipline at every level.

3.2 The meta-rank must have extensional content

It is easy to add a large natural parameter to a definition without proving that changing the parameter changes the result. That would be mathematically unpersuasive. The proposed successor rank therefore carries the preceding rank at coordinate zero, and the final autonomous lift explicitly exceeds that coordinate. This yields the theorem `rankLift_step_lt` for every rank, base transformer, seed family, and numerical input.

3.3 The previous contender must enter as a construction, not a constant

The new definition uses

$$\texttt{family13}(n) = F_{13}(F_{13}(n)),$$

so $\texttt{family13}(42) = C_{13}$ by definitional equality. It never mentions `contender_13` in the definition modules. The old construction controls the seed, the meta-rank, and the schedule rank. This is a qualitative reuse of its growth mechanism, not a padded successor such as $C_{13} + 1$.

3.4 The closure must remain executable

Every new recursion is either structural recursion on a W-tree or primitive recursion on a natural number. There is no search over an undecidable predicate, no Busy Beaver, no classical choice, and no semantic oracle. The normalization of dynamic-kind programs is inherited from the repository's certified total normalizer.

3.5 The final layer should expose a direct program bound

A pure meta-amplifier could dominate the seed without making it transparent which object-language programs are below the final result. We therefore close the rank-diagonal family with one explicit dynamic-kind family reflection. This makes the final program bound a one-line consequence of finite maximum membership.

4 The existing progressivity interface

Let

$$\mathsf{Family} = \mathbb{N} \to \mathbb{N} \to \mathbb{N}$$

be the type of countable families of unary functions. A transformer is

$$\mathsf{Transformer} = \mathsf{Family} \to \mathsf{Family}.$$

BusyLean defines

$$\mathsf{Progressive}(T) :\Longleftrightarrow \forall s, i, o, n \in \mathbb{N},\ i \le n \Longrightarrow s(i,n) < T(s)(o,n).$$

The output coordinate o is arbitrary. Thus a progressive transformer strictly dominates every seed coordinate visible at the current numerical budget.

Two generic operations are already available.

Splicing. Given $T_i : \mathsf{Transformer}$,

$$\texttt{spliceTransformers}(i \mapsto T_i)(s)(o,n) = T_o(s)(0,n).$$

If every T_i is progressive, the splice is progressive.

Autonomous lifting. `liftTransformer` runs a transformer along the repository's universal W-tree `index`. The theorem `transformer_lt_lift_at` follows an explicit path from a leaf through `treesOfDepth 0` into `index`; it proves that a visible output of the original transformer is below every output of its lift. This is stronger than mere preservation of progressivity and is the engine behind strict meta-rank separation.

These lemmas are important because they make the new development independent of the internal semantics of System F_ω. Once the base reflection is proved progressive, the rest is generic order theory over structurally recursive operators.

5 One level higher: meta-transformers

Definition 5.1 (Meta-transformer)**.** A meta-transformer is a function

$$A : \mathsf{MetaTransformer} = \mathsf{Transformer} \to \mathsf{Transformer}.$$

It preserves progressivity when

$$\mathsf{PreservesProgressive}(A) :\Longleftrightarrow \forall T,\ \mathsf{Progressive}(T) \Longrightarrow \mathsf{Progressive}(A(T)).$$

The central tree operation is the next-type analogue of `operatorStage`.

Definition 5.2 (Meta-stage)**.** For a meta-transformer A, base transformer B, and W-tree a, define

$$\begin{aligned} \mathsf{metaStage}(A, B, \mathsf{leaf}) &= A(B), \\ \mathsf{metaStage}(A, B, \mathsf{node}(f)) &= A\big(\mathsf{splice}(i \mapsto \mathsf{metaStage}(A, B, f(i)))\big)\,. \end{aligned}$$

This is structural recursion on a. At a node, every child contributes a transformer, those child transformers are spliced into a single transformer, and the meta-transformer is applied once.

Theorem 5.3 (Progressivity of the meta-stage)**.** *If A preserves progressivity and B is progressive, then* $\mathsf{metaStage}(A, B, a)$ *is progressive for every W-tree a.*

Proof. By structural induction on a. At a leaf, apply $\mathsf{PreservesProgressive}(A)$ directly to B. At a node, every recursively constructed child transformer is progressive by the induction hypotheses. Their splice is progressive by `spliceTransformers_progressive`; applying A preserves that property. □

The Lean theorem is `metaStage_progressive`; its proof has exactly these two cases and does not inspect any object-language term.

6 Anchored successor ranks

A naive hierarchy could set $A_{r+1}(B) = \mathsf{metaStage}(A_r, B, a)$. This is a natural amplifier, but a proof that rank $r + 1$ is extensionally larger than rank r would then have to locate a suitable leaf inside the schedule. The proposed hierarchy makes that relation explicit.

Definition 6.1 (Anchored rank step)**.** Fix a schedule a. For a meta-transformer A, define a family of transformers

$$U_0 = A(B), \qquad U_{j+1} = \mathsf{metaStage}(A, B, a).$$

Then set

$$\mathsf{rankStep}_a(A)(B) = \mathsf{liftTransformer}(\mathsf{splice}(U)).$$

Coordinate zero is the anchor. The positive coordinates repeat the full schedule amplification; repeating the same transformer is intentional, since the splice's purpose is to make the old rank and the scheduled rank jointly visible to the autonomous lift.

Lemma 6.2 (Preservation)**.** *If A preserves progressivity, then so does* $\mathsf{rankStep}_a(A)$*.*

Proof. Let B be progressive. The anchor $A(B)$ is progressive. The full scheduled meta-stage is progressive by the preceding theorem. Hence every member of U is progressive, the splice is progressive, and autonomous lifting preserves progressivity. □

Definition 6.3 (Finite meta-rank hierarchy)**.** For a fixed schedule a, define

$$\mathsf{rankLift}_a(0) = \mathsf{liftTransformer},$$
$$\mathsf{rankLift}_a(r + 1) = \mathsf{rankStep}_a(\mathsf{rankLift}_a(r)).$$

The hierarchy is primitive recursive on r. The preservation lemma gives `rankLift_preserves` by immediate induction.

Theorem 6.4 (Strict successor-rank separation)**.** *If B is progressive, then for every seed s, rank r, input n, and output coordinate o,*

$$\mathsf{rankLift}_a(r)(B)(s)(0, n) < \mathsf{rankLift}_a(r + 1)(B)(s)(o, n).$$

Proof. At successor rank, the transformer supplied to `liftTransformer` is a splice whose coordinate zero is exactly $\mathsf{rankLift}_a(r)(B)$. The splice is progressive by the preceding lemmas. Apply `transformer_lt_lift_at` at visible coordinate zero. No semantic monotonicity is used. □

This theorem is the reason for the anchored design. The rank parameter is not merely a recursion counter: each increment creates a formally larger canonical value. In the supplied Lean file it is `rankLift_step_lt`.

7 Family-aware dynamic-kind reflection

Contender 13's `reflectFd` has a unary oracle. The repository already contains a computable pairing convention for passing a family through a unary oracle:

$$\mathsf{code}(i, m) = 2^i(2m + 1),$$

with

$$\texttt{packFamily}\ fs\ (2^i(2m + 1)) = fs(i, m).$$

The inverse extracts the 2-adic valuation and the remaining odd component. The relevant inverse theorem, `packFamily_code`, is already formalized for Contenders 9 and 11.

Definition 7.1 (Dynamic-kind family reflection)**.** For $fs : \mathsf{Family}$, let

$$\mathsf{R}_{\mathrm{Fd}}(fs, n) = n + 1 + \max\Big(\max_{i \le n} fs(i, n),\ \max\{\texttt{Fd.fvalue}(\texttt{packFamily}(fs), n, t) : t \in \texttt{Fd.ctmsUpTo}(n)\}\Big).$$

The corresponding Lean definition is `Fd.reflectFamily`. It is a finite calculation once its arguments are known: `ctmsUpTo n` is a finite list, `fvalue` is total by the repository's normalization theorem, and both maxima recurse over finite data.

Three direct inequalities follow.

Lemma 7.2 (Visible oracle bound)**.** *If $i \le n$, then*

$$fs(i, n) < \mathsf{R}_{\mathrm{Fd}}(fs, n).$$

Lemma 7.3 (Program bound)**.** *If $t \in$ `Fd.ctmsUpTo`(n), then*

$$\texttt{Fd.fvalue}(\texttt{packFamily}(fs), n, t) < \mathsf{R}_{\mathrm{Fd}}(fs, n).$$

Lemma 7.4 (Input bound)**.** *For every n,*

$$n < \mathsf{R}_{\mathrm{Fd}}(fs, n).$$

Define

$$T_{\mathrm{Fd}}(fs)(o, n) = \mathsf{R}_{\mathrm{Fd}}(fs, n).$$

The visible oracle bound immediately proves $\mathsf{Progressive}(T_{\mathrm{Fd}})$. The output coordinate is ignored, but keeping it in the transformer interface allows the generic rank machinery to use the same APIs as Contender 12.

8 The proposed number

We can now state the construction compactly.

Definition 8.1 (Previous family)**.**

$$\texttt{family13}(n) = F_{13}(F_{13}(n)).$$

In particular, $\texttt{family13}(42) = C_{13}$ by unfolding definitions.

Definition 8.2 (Diagonal rank)**.**

$$r_{14}(n) = \texttt{family13}(n) + 1.$$

The extra one ensures a positive rank even at small inputs and is harmless at the scales involved.

Definition 8.3 (Diagonal schedule)**.** Let `tree12` be Contender 12's autonomous schedule. Define

$$a_{14}(n) = \texttt{Autonomous.realize}\Big(\texttt{Autonomous.hyper}\big(r_{14}(n), \texttt{Autonomous.encode}(\texttt{tree12})\big)\Big).$$

This is the second diagonal axis: the number of higher derivatives above `tree12` is not 42 or any other fixed numeral, but the previous construction evaluated at the current input.

Let the seed family repeat the previous family at every coordinate:

$$s_{13}(i, m) = \texttt{family13}(m).$$

Define the rank-diagonal family

$$H_{14}(n) = \mathsf{rankLift}_{a_{14}(n)}(r_{14}(n))(T_{\mathrm{Fd}})(s_{13}).$$

In Lean this is `rankFamily14 n`.

Finally define

$$F_{14}(n) = \mathsf{R}_{\mathrm{Fd}}(H_{14}(n), n),$$

and

$$\boxed{C_{14} = F_{14}(F_{14}(42)).}$$

The dependency graph is shown in Figure 1.

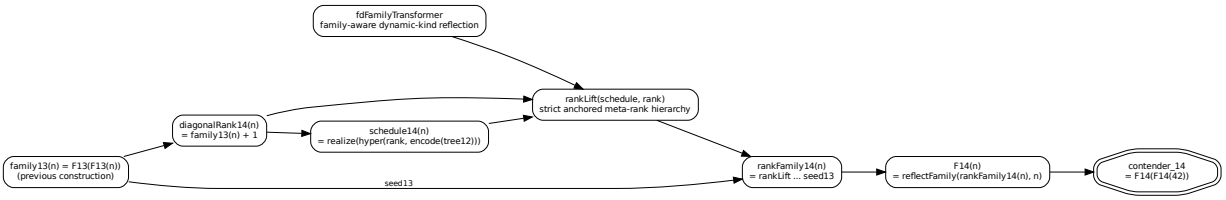


Figure 1: Data flow of the proposed Contender 14. The previous construction controls the seed, meta-rank, and autonomous schedule; the complete ranked family is closed by one more family-aware dynamic-kind reflection.

9 Formal comparison with Contender 13

The central comparison is deliberately short because the generic machinery carries the complexity.

Lemma 9.1 (Seed below ranked family)**.** *For every n,*

$$\textit{family13}(n) < H_{14}(n)(0, n).$$

Proof. The transformer T_{Fd} is progressive. Every finite $\mathsf{rankLift}$ built from a progressive base is progressive. Apply that property to seed coordinate zero, output coordinate zero, and input n:

$$s_{13}(0, n) < \mathsf{rankLift}_{a_{14}(n)}(r_{14}(n))(T_{\mathrm{Fd}})(s_{13})(0, n).$$

The left side unfolds to `family13 n`; the right side unfolds to `rankFamily14 n 0 n`. □

Lemma 9.2 (Ranked family below the closing reflection)**.** *For every n and every $i \le n$,*

$$H_{14}(n)(i,n) < F_{14}(n).$$

Proof. This is the visible oracle bound for R_{Fd}, because $F_{14}(n) = \mathsf{R}_{\mathrm{Fd}}(H_{14}(n), n)$. □

Theorem 9.3 (Pointwise dominance)**.** *For every n,*

$$\mathit{family13}(n) < F_{14}(n).$$

Proof. Compose the two preceding strict inequalities at coordinate zero. □

This is stronger than the one-off statement $C_{13} < C_{14}$: it proves that the full function underlying Contender 13 is pointwise below the new function.

The input bound for Contender 13 gives

$$n < F_{13}(n) < F_{13}(F_{13}(n)) = \texttt{family13}(n).$$

Combining this with Theorem 9.3 yields

$$n < F_{14}(n).$$

At $n = 42$,

$$\begin{aligned} C_{13} &= \texttt{family13}(42) \\ &< F_{14}(42) \\ &< F_{14}(F_{14}(42)) \\ &= C_{14}. \end{aligned}$$

Thus the required comparison is obtained.

Theorem 9.4 (Proposed Lean comparison)**.** *The supplied source states and proves*

$$\mathit{contender_13_lt_contender_14} : \mathit{contender_13} < \mathit{contender_14}.$$

Caution 9.5. The theorem source is complete and contains no placeholder, but this document does not label it "kernel checked" because Lean was unavailable in the artifact-building sandbox. The exact command needed for kernel replay is included in `scripts/check14.py`.

10 Additional formal margins

The proposal includes more than the minimum comparison theorem.

10.1 Every successor rank is strictly larger

Theorem 6.4 is formalized as `rankLift_step_lt`. This is a local theorem at every numerical input, not an asymptotic heuristic. It demonstrates that setting the rank to $r_{14}(n)$ has genuine order-theoretic meaning.

10.2 The diagonal rank escapes every fixed finite rank

The input-growth theorem for `family13` gives, whenever $k \le n$,

$$k \le n < \texttt{family13}(n) < \texttt{family13}(n) + 1 = r_{14}(n).$$

The included Lean theorem is `fixed_rank_lt_diagonalRank14`. Consequently no fixed finite meta-rank captures the diagonal rank function on all sufficiently large inputs. This statement concerns the rank index itself. Combined with strict successor-rank separation, it explains why the construction is not equivalent to choosing a large but fixed stack in advance.

10.3 Final-layer program domination

For every $t \in \texttt{Fd.ctmsUpTo}(n)$,

$$\texttt{Fd.fvalue}(\texttt{packFamily}(H_{14}(n)), n, t) < F_{14}(n).$$

This is `fvalue_lt_F14`. Substituting $n = F_{14}(42)$ gives `fvalue_lt_contender_14`. It says that the final contender exceeds every enumerated dynamic-kind program at the final syntax budget, when that program is allowed to query the entire rank-diagonal family through the packed oracle.

11 Why the increase is significant

"Significant" should not be justified by rhetoric about enormous decimal expansions. Four structural facts distinguish Contender 14 from Contender 13.

A stronger oracle interface. Contender 13 reflects over one unary function `family12`. The new base reflection exposes a countable family to dynamic-kind programs. Programs can compute the pairing code and query arbitrary family coordinates. The explicit prefix maximum simultaneously gives the comparison proof access to all visible coordinates.

A new type level of amplification. Contender 12 operates on transformers. Contender 14 treats operations on transformers as first-class mathematical objects, iterates them on W-trees, and then iterates an anchored promotion by natural rank. This is not a larger syntax budget for the same finite maximum; it is a new closure hierarchy.

The rank is previous-number controlled. A fixed rank, however elaborate, is still a fixed finite composition. Here $r_{14}(n) = \texttt{family13}(n) + 1$. The rank escapes every fixed finite bound, and each successor rank has a formal strict separation theorem.

The schedule is independently diagonalized. The autonomous schedule is also raised through $r_{14}(n)$ higher derivatives above `tree12`. This provides a second axis of growth rather than using the rank merely to repeat one fixed schedule. No exact ordinal identity is asserted for this syntax; its role is to supply a well-founded, computable family of increasingly nested tree schedules.

A final reflective closure. The rank-diagonal family is not simply read at one coordinate. It becomes the oracle family for another complete dynamic-kind reflection. Thus the final result directly dominates visible ranked values and all enumerated programs relative to that ranked oracle.

These features give both a formal margin and a qualitative one. The formal margin is Theorem 9.3. The qualitative margin is the move from one dynamic-kind reflection to a previous-number-indexed hierarchy of strictly separated meta-reflections, followed by another dynamic-kind reflection.

12 Computability and constructiveness

The number is not practically evaluable, but practical evaluability is not the contest's criterion. The correct question is whether the definition denotes a total recursive natural number without nonconstructive data.

Proposition 12.1 (Computability in principle)**.** *Assuming the totality theorems already formalized for the repository's object-language evaluator,* `contender_14` *is a computable natural number.*

Construction audit. Each component is total for a direct reason.

1. `Fd.ctmsUpTo n` is generated by primitive recursion and is finite.
2. `Fd.fvalue` invokes the repository's certified normalizer on a term accepted by its type checker; otherwise it returns zero.
3. `packFamily` uses primitive recursive arithmetic, including the 2-adic valuation with a structurally justified decreasing argument.
4. `Fd.reflectFamily` takes maxima over finite lists and a finite prefix of the oracle family.
5. `metaStage` is structural recursion on a W-tree.
6. `rankLift` is primitive recursion on a natural number.
7. `Autonomous.hyper` is primitive recursion on the derivative level; `Autonomous.realize` is structural recursion on the resulting notation.
8. `F14` composes these total functions, and `contender_14` applies `F14` twice to 42.

No definition requires deciding an arbitrary proposition, choosing an element of an arbitrary inhabited type, consulting an external oracle, or postulating termination. Therefore the closed term computes a natural number in principle. □

The source-level audit strips comments and strings and checks the definition modules for the following prohibited tokens:

```
noncomputable   partial  unsafe  opaque            extern
implemented_by  sorry    admit   Classical.choice  native_decide
```

It also checks that no earlier contender constant occurs in the new definition modules. That audit passed. It is intentionally not represented as a substitute for kernel checking.

13 Axiom and dependency discipline

The focused audit file performs two kernel-level checks when replayed.

First, it requests `#print axioms` for the final definition and the principal theorems. The build harness accepts only `propext` and `Quot.sound`, matching the repository policy.

Second, it computes the transitive constant closure of `BusyLean.contender_14` and rejects any occurrence of `contender_5` through `contender_13`. This is stronger than a text search: it traverses the types and values of every reachable declaration. The expected closure contains the previous *constructions*—notably `F13`, `F12`, the normalizers, and the W-tree infrastructure—but not the previous contender constants.

The distinction matters. A definition such as `contender_13` + 1 would pass a numerical comparison but violate the spirit and explicit rules of the contest. By contrast, `family13`$(n) = F_{13}(F_{13}(n))$ exposes the reusable growth function at arbitrary input and permits a pointwise theorem.

14 Implementation layout

The overlay adds three library modules.

`HyperReflection.lean`
: Defines meta-transformers, tree iteration, anchored rank promotion, finite rank lifting, progressivity preservation, and strict rank separation.

`Contender14.lean`
: Defines family-aware dynamic-kind reflection, the previous family, diagonal rank, diagonal schedule, ranked oracle family, `F14`, and `contender_14`.

`Comparison14.lean`
: Proves the direct reflection inequalities, progressivity, pointwise dominance, input growth, the main comparison, rank escape, and program bounds.

The package also adds `test/Audit14.lean` and `scripts/check14.py`. The installer appends the three imports to `BusyLean.lean` without modifying the existing contender files.

The build sequence is intentionally staged:

$$\texttt{BusyLean.Comparison13} \longrightarrow \texttt{BusyLean.Contender14} \longrightarrow \texttt{BusyLean.Comparison14}.$$

The first phase is the accepted baseline. The second is measured against the 15-second definition budget; the third against the 60-second proof budget. Both the repository's existing audit and the focused Contender 14 audit then run under `lake env lean`.

15 Validation status and likely proof obligations

The lack of a Lean executable in the artifact-building environment requires a clear distinction between source completeness and kernel validation.

15.1 What was checked

The source audit verified balanced delimiters after stripping nested comments and strings; absence of prohibited implementation tokens; absence of earlier contender constants in the new definition modules; line-length and whitespace hygiene; and deterministic file hashes. The mathematical proof structure was cross-checked against the exact signatures of these audited repository lemmas:

```
spliceTransformers_progressive  liftTransformer_progressive
transformer_lt_lift_at          W.le_prefixMax
W.le_maximum
```

15.2 What kernel replay must confirm

Lean replay must confirm elaboration of the recursive theorem `metaStage_progressive`; reduction of the anchored coordinate in `rankLift_step_lt`; definitional equalities connecting `family13 42` with `contender_13`; and the expected axiom closure. These are ordinary elaboration obligations rather than gaps in the mathematical argument, but they remain real acceptance conditions.

15.3 Performance risk

The definitions are deliberately written so that type checking does not normalize the enormous ranks. The rank appears as a symbolic argument to primitive recursion, and proofs use generic induction theorems. Nevertheless, Lean's elaborator may unfold more than expected while resolving definitional equalities, particularly in the strict-rank theorem's final simplification. The included build harness measures the exact cost. If optimization is needed, the first response should be to replace broad simplification with two explicit `change` steps, not to weaken the construction.

16 Informal growth interpretation—and its limits

It is tempting to assign a named proof-theoretic ordinal to every contender. That temptation should be resisted here. The exact strength of the repository's dynamic-kind language, its interaction with packed external oracles, and the higher-type autonomous reflection hierarchy has not been calibrated in a published ordinal analysis. The present formal result does not need such a calibration.

A safe qualitative description is available. Contender 13 takes one bounded maximum over dynamic-kind programs relative to a previous oracle. Contender 14 uses the same language as a progressive transformer over a countable family, builds a strictly increasing hierarchy of operations on that transformer, selects a rank larger than the previous contender's family value, independently raises the tree schedule by the same value, and then reflects once more over the whole result. Any exact fast-growing-hierarchy assignment would have to model all of those closures. The direct Lean inequalities are therefore the primary mathematics; ordinal terminology is at most a guide for future work.

17 Potential refinements after kernel replay

The current proposal aims for a strong construction with a compact proof surface. Several refinements could increase either mathematical clarity or growth strength.

17.1 Transitive rank comparison

The supplied theorem compares rank r with rank $r + 1$. An induction can package this as strict monotonicity in the rank argument at canonical coordinate zero:

$$r < s \Longrightarrow \mathsf{rankLift}_a(r)(B)(s_0)(0, n) < \mathsf{rankLift}_a(s)(B)(s_0)(0, n).$$

This would combine directly with `fixed_rank_lt_diagonalRank14` to show that the diagonal ranked family strictly dominates every fixed rank, pointwise beyond the rank index.

17.2 Coordinate-sensitive positive branches

The present positive coordinates of `rankStep` repeat the same scheduled meta-stage. One can instead let coordinate $j + 1$ use a schedule transformed by j, for example an additional autonomous derivative or a selected child of the schedule. The preservation proof is unchanged provided every branch is progressive. This would make the countable splice carry genuinely distinct scheduled amplifiers.

17.3 A hierarchy of dynamic universes

Contender 13 uses one predicative dynamic kind whose elements range over closed dyn-free kinds. A natural language-level extension is a finite tower $\mathsf{dyn}_0, \ldots, \mathsf{dyn}_k$, where dyn_{j+1} packages values from

kinds containing lower dynamic levels but not itself. Diagonalizing over k would strengthen the object language rather than only the surrounding amplifier. Such an extension would require a new normalization proof and is therefore a larger research project.

17.4 Internal tree generation

The current dynamic-kind language has no set-theoretic W-tree value that can be returned to Lean. A language combining the dynamic universe with a strictly positive tree universe could generate its own meta-reflection schedules. The main challenge is a predicative semantic model supporting large elimination without collapsing into an inconsistent type-in-type system.

17.5 Exact ordinal analysis

A worthwhile theoretical project is to isolate a transparent subsystem of the rank-reflection hierarchy and assign an ordinal notation with proved upper and lower bounds. The anchor construction may be useful because it exposes a canonical embedding from each rank into the next. Any such analysis should be kept separate from the Lean comparison theorem until the semantic assumptions are fully explicit.

18 Reproduction protocol

Starting from the audited BusyLean commit, unpack this archive and run:

```
cd BusyLean_Contender14_RankDiagonal
./install.sh /path/to/BusyLean
cd /path/to/BusyLean
python3 scripts/check14.py
```

The installer is idempotent. It copies the three Lean modules, the focused audit, and the build script, then appends imports to `BusyLean.lean` only when they are absent. The same changes are available as a unified patch for manual review.

A successful run should establish all of the following:

1. the accepted baseline through `Comparison13` builds;
2. the new definition module builds within 15 seconds;
3. the comparison module builds within 60 seconds;
4. the repository audit still succeeds;
5. the Contender 14 axiom outputs contain no forbidden axiom;
6. the transitive closure of `contender_14` contains no earlier contender constant.

The actual timing output and axiom report should be committed alongside any integration pull request. If the base repository has moved beyond the audited commit, review the current contender first rather than mechanically applying the patch.

19 Conclusion

Contender 14 is designed to be a genuine next step in the architecture of BusyLean. It does not rely on adding one, increasing a fixed syntax budget, or reusing the previous constant. It upgrades the dynamic-kind oracle to a family, lifts autonomous reflection to meta-transformers, makes every rank increment formally strict, diagonalizes both rank and schedule by the previous construction, and closes the result with another dynamic-kind reflection.

The formal comparison has a clean spine:

$$\texttt{family13}(n) < \texttt{rankFamily14}(n)(0, n) < F_{14}(n),$$

followed by input growth and the usual outer diagonal. The strongest claims are therefore exact inequalities, not informal ordinal labels. The package is ready for the one remaining acceptance step: replay under the repository's pinned Lean 4.34.0 toolchain and measurement against the contest budgets.

A Complete Lean source: meta-reflection hierarchy

```
import BusyLean.MetaReflectionFacts

/-!
# Reflection lifted one level above transformers

`Transformer = Family → Family` is the level used by Contender 12.  This file
adds a total, structural hierarchy at the next level:

* a `MetaTransformer` maps transformers to transformers;
* `metaStage` iterates a meta-transformer along a well-founded `WTree Nat`;
* `rankLift schedule r` recursively promotes autonomous lifting through `r`
  meta-levels, using `schedule` at every positive level.

The proofs use only the explicit progressivity interface from
`BusyLean.MetaReflectionFacts`.  No monotonicity assumption is made.
-/

namespace BusyLean

open BigNumber (WTree)
open BusyLean.W

/-- An operator on family transformers. -/
abbrev MetaTransformer := Transformer → Transformer

/-- A meta-transformer preserves the strict pointwise progressivity invariant. -/
def PreservesProgressive (amp : MetaTransformer) : Prop :=
  ∀ op, Progressive op → Progressive (amp op)

/-- Iterate a meta-transformer along a well-founded tree.

At a leaf, apply `amp` once.  At a node, recursively construct one transformer
from every child, splice that family of transformers, and apply `amp` to the
splice.  This is the exact next-type analogue of `operatorStage`.
-/
def metaStage (amp : MetaTransformer) (base : Transformer) :
    WTree Nat → Transformer
  | .leaf => amp base
```

```
  | .node f =>
      amp (spliceTransformers (fun i => metaStage amp base (f i)))

/-- Tree iteration of a progressivity-preserving meta-transformer remains
progressive. -/
theorem metaStage_progressive {amp : MetaTransformer}
    (hamp : PreservesProgressive amp) {base : Transformer}
    (hbase : Progressive base) :
    (a : WTree Nat) → Progressive (metaStage amp base a)
  | .leaf => hamp base hbase
  | .node f =>
      hamp
        (spliceTransformers (fun i => metaStage amp base (f i)))
        (spliceTransformers_progressive
          (fun i => metaStage_progressive hamp hbase (f i)))

/-- Branches used by one anchored rank promotion.

Coordinate zero preserves the preceding rank verbatim.  Every positive
coordinate carries the full schedule iteration.
-/
def rankBranches (schedule : WTree Nat) (amp : MetaTransformer)
    (base : Transformer) : Nat → Transformer
  | 0 => amp base
  | _ + 1 => metaStage amp base schedule

/-- Every anchored branch is progressive when the meta-transformer preserves
progressivity and the base is progressive. -/
theorem rankBranches_progressive (schedule : WTree Nat)
    {amp : MetaTransformer} (hamp : PreservesProgressive amp)
    {base : Transformer} (hbase : Progressive base) :
    ∀ i, Progressive (rankBranches schedule amp base i)
  | 0 => hamp base hbase
  | _ + 1 => metaStage_progressive hamp hbase schedule

/-- One anchored promotion of a meta-transformer.

The branch family is spliced and then autonomously lifted.  The final lift
makes the new rank strictly exceed the old rank at its canonical coordinate,
while retaining the schedule-driven amplification on positive coordinates.
-/
def rankStep (schedule : WTree Nat) (amp : MetaTransformer) :
    MetaTransformer :=
  fun base =>
    liftTransformer
      (spliceTransformers (rankBranches schedule amp base))

/-- Anchored promotion preserves progressivity. -/
theorem rankStep_preserves (schedule : WTree Nat) {amp : MetaTransformer}
    (hamp : PreservesProgressive amp) :
    PreservesProgressive (rankStep schedule amp) := by
  intro base hbase
  exact liftTransformer_progressive
    (spliceTransformers_progressive
      (rankBranches_progressive schedule hamp hbase))

/-- The rank hierarchy of meta-level autonomous lifting.

Rank zero is the autonomous lift from Contender 12.  Every successor is an
anchored promotion of the preceding meta-transformer.  The rank is a plain
```

```
natural number, so the hierarchy is executable in principle.
-/
def rankLift (schedule : WTree Nat) : Nat → MetaTransformer
  | 0 => liftTransformer
  | r + 1 => rankStep schedule (rankLift schedule r)

/-- Every finite rank preserves progressivity. -/
theorem rankLift_preserves (schedule : WTree Nat) :
    ∀ r, PreservesProgressive (rankLift schedule r)
  | 0 => fun base hbase => liftTransformer_progressive hbase
  | r + 1 =>
      rankStep_preserves schedule (rankLift_preserves schedule r)

/-- Convenient pointwise form of `rankLift_preserves`. -/
theorem rankLift_progressive (schedule : WTree Nat) (r : Nat)
    {base : Transformer} (hbase : Progressive base) :
    Progressive (rankLift schedule r base) :=
  rankLift_preserves schedule r base hbase

/-- Every successor rank strictly exceeds the preceding rank at canonical
coordinate zero.  This makes the rank parameter extensionally meaningful,
not merely a syntactic counter. -/
theorem rankLift_step_lt (schedule : WTree Nat) (r : Nat)
    {base : Transformer} (hbase : Progressive base) (seed : Family)
    {o n : Nat} :
    rankLift schedule r base seed 0 n <
      rankLift schedule (r + 1) base seed o n := by
  change
    rankBranches schedule (rankLift schedule r) base 0 seed 0 n <
      liftTransformer
        (spliceTransformers
          (rankBranches schedule (rankLift schedule r) base)) seed o n
  exact transformer_lt_lift_at
    (spliceTransformers_progressive
      (rankBranches_progressive schedule
        (rankLift_preserves schedule r) hbase))
    seed (Nat.zero_le n)

end BusyLean
```

B Complete Lean source: Contender 14 definition

```
import BusyLean.Contender13
import BusyLean.Contender9
import BusyLean.Autonomous
import BusyLean.HyperReflection

/-!
# Contender 14: bi-diagonal rank reflection over dynamic-kind System Fω

Contender 13 performs one unary-oracle reflection over dynamic-kind System Fω.
The present construction strengthens all three axes at once.

1. `Fd.reflectFamily` gives that language a countable oracle family, encoded by
   `packFamily`, while retaining an explicit maximum over all visible family
   coordinates.
2. `rankLift` raises autonomous reflection through a finite hierarchy of
   meta-levels.  The rank at input `n` is `family13 n + 1` rather than a fixed
```

```
   numeral.
3. The W-tree schedule is also diagonal: the number of higher derivatives
   stacked above Contender 12's tree is `family13 n + 1`.

The ranked family is finally reflected once more by dynamic-kind System Fω.
Thus `F14 n` strictly exceeds both Contender 13's family at `n` and every
visible coordinate of the entire rank-diagonal family.

**Do not evaluate `contender_14`.**
-/

namespace BusyLean.Fd

/-- Dynamic-kind System Fω reflection with a countable oracle family.

The family is exposed to object-language programs through the computable code
`packFamily fs (2^i * (2*m+1)) = fs i m`.  The explicit prefix maximum is what
makes progressivity independent of any semantic monotonicity of programs.
-/
def reflectFamily (fs : W.Family) (n : Nat) : Nat :=
  n + 1 + BigNumber.maxN
    (W.prefixMax (fun i => fs i n) n)
    (BigNumber.maximum
      (fvalue (BusyLean.packFamily fs) n) (ctmsUpTo n))

end BusyLean.Fd

namespace BusyLean

open BigNumber (WTree)
open BusyLean.W

/-- Contender 13's construction as a function of its input:
`family13 42 = contender_13`. -/
def family13 (n : Nat) : Nat := F13 (F13 n)

/-- The previous construction determines both the meta-rank and schedule rank. -/
def diagonalRank14 (n : Nat) : Nat := family13 n + 1

/-- A schedule obtained by stacking a previous-number-controlled number of
higher critical derivatives above Contender 12's already autonomous tree. -/
def schedule14 (n : Nat) : WTree Nat :=
  Autonomous.realize
    (Autonomous.hyper (diagonalRank14 n) (Autonomous.encode tree12))

/-- One family-aware dynamic-kind reflection step, viewed as a transformer. -/
def fdFamilyTransformer : Transformer :=
  fun fs _ n => Fd.reflectFamily fs n

/-- The seed family repeats Contender 13's construction at every coordinate. -/
def seed13 : W.Family := fun _ => family13

/-- The rank-diagonal family at input `n`.

Both the meta-rank and the autonomous schedule are controlled by
`family13 n + 1`; neither is a fixed finite amplification.
-/
def rankFamily14 (n : Nat) : W.Family :=
  rankLift (schedule14 n) (diagonalRank14 n) fdFamilyTransformer seed13
```

```
/-- The final dynamic-kind reflection over the complete rank-diagonal family. -/
def F14 (n : Nat) : Nat := Fd.reflectFamily (rankFamily14 n) n

/-- **Contender 14.** Never evaluate it. -/
def contender_14 : Nat := F14 (F14 42)

end BusyLean
```

C Complete Lean source: comparison theorems

```
import BusyLean.Contender14
import BusyLean.Comparison13

/-! # Contender 14: comparison theorems -/

namespace BusyLean.Fd

/-- A bound on the reflected finite maximum is strictly exceeded. -/
theorem lt_reflectFamily {fs : W.Family} {n m : Nat}
    (h : m ≤ BigNumber.maxN
      (W.prefixMax (fun i => fs i n) n)
      (BigNumber.maximum
        (fvalue (BusyLean.packFamily fs) n) (ctmsUpTo n))) :
    m < reflectFamily fs n := by
  show m < n + 1 + _
  have hM : ∀ M : Nat, M + 1 ≤ n + 1 + M := fun M => by
    rw [Nat.add_comm M 1]
    exact Nat.add_le_add_right (Nat.le_add_left 1 n) M
  exact Nat.lt_of_le_of_lt h
    (Nat.lt_of_lt_of_le (Nat.lt_succ_self _) (hM _))

/-- Every visible oracle-family coordinate is exceeded. -/
theorem oracle_lt_reflectFamily (fs : W.Family) {i n : Nat}
    (hi : i ≤ n) : fs i n < reflectFamily fs n :=
  lt_reflectFamily
    (Nat.le_trans (W.le_prefixMax (fun j => fs j n) hi)
      (BigNumber.le_maxN_left _ _))

/-- Every enumerated dynamic-kind program is exceeded. -/
theorem fvalue_lt_reflectFamily (fs : W.Family) {n : Nat} {t : CTm}
    (ht : t ∈ ctmsUpTo n) :
    fvalue (BusyLean.packFamily fs) n t < reflectFamily fs n :=
  lt_reflectFamily
    (Nat.le_trans (W.le_maximum (fvalue (BusyLean.packFamily fs) n) ht)
      (BigNumber.le_maxN_right _ _))

/-- The numerical input itself is exceeded. -/
theorem input_lt_reflectFamily (fs : W.Family) (n : Nat) :
    n < reflectFamily fs n :=
  Nat.lt_of_lt_of_le (Nat.lt_succ_self n) (Nat.le_add_right _ _)

end BusyLean.Fd

namespace BusyLean

/-- Family-aware dynamic-kind reflection is a progressive transformer. -/
theorem fdFamilyTransformer_progressive :
    Progressive fdFamilyTransformer := by
```

```
  intro seed i o n hi
  exact Fd.oracle_lt_reflectFamily seed hi

/-- Contender 13's family lies below the rank-diagonal family at every input. -/
theorem family13_lt_rankFamily14 (n : Nat) :
    family13 n < rankFamily14 n 0 n := by
  change seed13 0 n <
    rankLift (schedule14 n) (diagonalRank14 n)
      fdFamilyTransformer seed13 0 n
  exact
    (rankLift_progressive (schedule14 n) (diagonalRank14 n)
      fdFamilyTransformer_progressive) seed13 0 0 n (Nat.zero_le n)

/-- Every visible coordinate of the rank-diagonal family is below `F14`. -/
theorem rankFamily14_lt_F14 {n i : Nat} (hi : i ≤ n) :
    rankFamily14 n i n < F14 n :=
  Fd.oracle_lt_reflectFamily (rankFamily14 n) hi

/-- Pointwise dominance over Contender 13's complete construction. -/
theorem family13_lt_F14 (n : Nat) : family13 n < F14 n :=
  Nat.lt_trans (family13_lt_rankFamily14 n)
    (rankFamily14_lt_F14 (n := n) (i := 0) (Nat.zero_le n))

/-- `F13` strictly exceeds its input. -/
theorem input_lt_F13 (n : Nat) : n < F13 n :=
  Fd.input_lt_reflectFd family12 n

/-- Contender 13's family strictly exceeds its input. -/
theorem input_lt_family13 (n : Nat) : n < family13 n :=
  Nat.lt_trans (input_lt_F13 n) (input_lt_F13 (F13 n))

/-- The diagonal meta-rank eventually lies above every fixed finite rank. -/
theorem fixed_rank_lt_diagonalRank14 {k n : Nat} (hkn : k ≤ n) :
    k < diagonalRank14 n :=
  Nat.lt_of_le_of_lt hkn
    (Nat.lt_trans (input_lt_family13 n) (Nat.lt_succ_self _))

/-- `F14` strictly exceeds its input. -/
theorem input_lt_F14 (n : Nat) : n < F14 n :=
  Nat.lt_trans (input_lt_family13 n) (family13_lt_F14 n)

/-- **The required comparison:** Contender 13 is strictly below Contender 14. -/
theorem contender_13_lt_contender_14 : contender_13 < contender_14 :=
  Nat.lt_trans (family13_lt_F14 42) (input_lt_F14 (F14 42))

/-- Every dynamic-kind program in the final pointwise reflection is exceeded. -/
theorem fvalue_lt_F14 {n : Nat} {t : Fd.CTm} (ht : t ∈ Fd.ctmsUpTo n) :
    Fd.fvalue (packFamily (rankFamily14 n)) n t < F14 n :=
  Fd.fvalue_lt_reflectFamily (rankFamily14 n) ht

/-- Final-instance form of the preceding program bound. -/
theorem fvalue_lt_contender_14 {t : Fd.CTm}
    (ht : t ∈ Fd.ctmsUpTo (F14 42)) :
    Fd.fvalue (packFamily (rankFamily14 (F14 42))) (F14 42) t <
      contender_14 :=
  fvalue_lt_F14 ht

/-- Transitive comparison with Contender 12. -/
theorem contender_12_lt_contender_14 : contender_12 < contender_14 :=
  Nat.lt_trans contender_12_lt_contender_13 contender_13_lt_contender_14
```

```
end BusyLean
```

D Focused audit source

```
import BusyLean.Comparison14
import Lean

/-! Focused audit for Contender 14. -/

open Lean Elab Command

/-- All constants reachable from `root` through types and values. -/
def constClosure14 (env : Environment) (root : Name) : NameSet := Id.run do
  let mut seen : NameSet := {}
  let mut stack := #[root]
  while stack.size > 0 do
    let c := stack.back!
    stack := stack.pop
    if seen.contains c then continue
    seen := seen.insert c
    let some ci := env.find? c | continue
    let body : Array Name := match ci with
      | .defnInfo v => v.value.getUsedConstants
      | .thmInfo v => v.value.getUsedConstants
      | .opaqueInfo v => v.value.getUsedConstants
      | _ => #[]
    stack := stack ++ ci.type.getUsedConstants ++ body
  return seen

/-- Fail if any forbidden constant is reachable from the root definition. -/
elab "#assert_not_in_closure14 " root:ident " : " bad:ident* : command => do
  let env ← getEnv
  let r ← liftCoreM <| realizeGlobalConstNoOverloadWithInfo root
  let cl := constClosure14 env r
  for b in bad do
    let n ← liftCoreM <| realizeGlobalConstNoOverloadWithInfo b
    if cl.contains n then
      throwError m!"{r} depends on forbidden constant {n}"
  logInfo m!"{r}: closure of {cl.size} constants avoids all {bad.size} forbidden names"

#print axioms BusyLean.contender_14
#print axioms BusyLean.contender_13_lt_contender_14
#print axioms BusyLean.family13_lt_F14
#print axioms BusyLean.fvalue_lt_contender_14
#print axioms BusyLean.rankLift_preserves
#print axioms BusyLean.rankLift_step_lt
#print axioms BusyLean.fixed_rank_lt_diagonalRank14

#assert_not_in_closure14 BusyLean.contender_14 :
  BusyLean.contender_13 BusyLean.contender_12 BusyLean.contender_11
  BusyLean.contender_10 BusyLean.contender_9 BusyLean.contender_8
  BusyLean.contender_7 BusyLean.contender_6 BusyLean.contender_5

#check @BusyLean.contender_13_lt_contender_14
#check @BusyLean.family13_lt_F14
#check @BusyLean.fvalue_lt_contender_14
#check @BusyLean.rankLift_preserves
```

```
#check @BusyLean.rankLift_step_lt
#check @BusyLean.fixed_rank_lt_diagonalRank14
```

References

[1] V. Reshetnikov and contributors, *BusyLean: name the biggest number, in Lean*, GitHub repository, audited commit `1ee50c7d97bbed57a60709f661453c3e213f4db3`, October 2026. https://github.com/VladimirReshetnikov/BusyLean

[2] C. Roux, *name-the-biggest-number*, GitHub repository. https://github.com/codyroux/name-the-biggest-number

[3] S. Aaronson, *Who Can Name the Bigger Number?* https://www.scottaaronson.com/writings/bignumbers.html

[4] J.-Y. Girard, *Interprétation fonctionnelle et élimination des coupures de l'arithmétique d'ordre supérieur*, Thèse d'État, Université Paris VII, 1972.

[5] J. C. Reynolds, Types, abstraction and parametric polymorphism, in *Information Processing 83*, pp. 513–523, 1983.

[6] J.-Y. Girard, Y. Lafont, and P. Taylor, *Proofs and Types*, Cambridge University Press, 1989.

[7] Lean developers, *The Lean Language Reference: Recursive Definitions and Inductive Types*, Lean 4 documentation. https://lean-lang.org/doc/reference/latest/Definitions/Recursive-Definitions/

[8] Lean developers, *Lean 4.34.0 release*, September 14, 2026. https://github.com/leanprover/lean4/releases/tag/v4.34.0